

PackageMaker

A GAP package for creating new GAP packages

1.0.0

24 March 2026

Max Horn

Max Horn

Email: mhorn@rptu.de

Homepage: <https://www.quendi.de/math>

Address: Fachbereich Mathematik

RPTU Kaiserslautern-Landau

Gottlieb-Daimler-Straße 48

67663 Kaiserslautern

Germany

Contents

1	PackageMaker	3
1.1	What is PackageMaker?	3
1.2	Before you start	3
1.3	Where should the generated package go?	4
1.4	A worked example	4
1.5	Important wizard choices	6
1.6	What gets generated?	7
1.7	What next?	7
	Index	9

Chapter 1

PackageMaker

1.1 What is PackageMaker?

PackageMaker is a GAP package that makes it easy to create new GAP packages, by providing a “wizard” function that asks a few questions about the new package, such as its name and authors; and from that creates a complete usable skeleton package. It optionally can set up several advanced features, including:

- a git repository for use on GitHub;
- a simple C or C++ kernel extension, including a build system;
- continuous integration and release automation using GitHub Actions;
- code coverage tracking;
- ... and more.

The generated package is meant as a starting point, not as a finished package ready for release. In particular, you should expect to edit `README.md`, `PackageInfo.g`, the generated tests and, depending on your choices, the documentation, CI workflows or kernel extension sources.

1.2 Before you start

A typical session looks like this:

- start GAP in a directory where it is OK to create a new package directory;
- load PackageMaker and run `PackageWizard()`;
- answer the wizard questions;
- inspect and edit the generated files;
- move the package to a directory in which GAP looks for packages, or run GAP with `--packagedirs`.

By default, the wizard creates the new package directory in the current working directory of your GAP session. That is often convenient for creating the package, but it does not automatically make the package loadable. See the next section for the different ways to make GAP find the generated package.

If you ask PackageMaker to create a git repository, git should have `user.name` and `user.email` configured. If they are missing, PackageMaker explains how to set them up, lets you retry, or keeps the generated package directory without creating a git repository.

1.3 Where should the generated package go?

The generated package directory must eventually live in a place where GAP searches for packages.

For normal day-to-day use, the best destination is usually `~/.gap/pkg`. You may have to create that directory first. Keeping personal packages there means they survive GAP upgrades and are separate from the main GAP installation.

Another natural choice is the `pkg` directory inside your main GAP installation. This is common when you built GAP from source in a directory that you control. The drawback is that when you later upgrade or reinstall GAP, it is easy to forget to copy over packages that you added there manually.

For quick testing, GAP 4.15 or later can be started with `gap --packagedirs .` to tell GAP to search the current directory directly for packages.

There are also more advanced alternatives:

- command line options such as `-l` (see **(Reference: Command Line Options)**) can change the configured GAP root directories (see **(Reference: GAP Root Directories)**) when starting GAP;
- `ExtendRootDirectories` (**(Reference: ExtendRootDirectories)**) and `ExtendPackageDirectories` (**(Reference: ExtendPackageDirectories)**) can add package search locations after GAP has already started;
- `SetPackagePath` (**(Reference: SetPackagePath)**) can be useful when you want to force loading one particular package from a specific path for a session.

These advanced options are useful to know about, but for most users the practical advice is simple: keep the package in `~/.gap/pkg` for normal use, and use `gap --packagedirs .` only as a quick-testing convenience when running GAP 4.15 or newer.

1.4 A worked example

The following transcript shows a complete example session. It uses GitHub, enables the generated workflows, chooses the MIT license, creates a C kernel extension, and enters one author / maintainer record.

1.4.1 PackageWizardInput

```
▷ PackageWizardInput(arg) (function)
```

Interactively create a package skeleton. You can abort by either answering “quit” or pressing `Ctrl-D`.

Example

```

gap> PackageWizard();
Welcome to the GAP PackageMaker Wizard.
I will now guide you step-by-step through the package
creation process by asking you some questions.

What is the name of the package? DemoPackage
Enter a short (one sentence) description of your package: A package used to demonstrate PackageMaker
Which license should the package use? MIT
Shall I prepare your new package for GitHub? [Y/n] y
Do you want to use GitHub Actions for automated tests and making releases? [Y/n] y
The release workflow updates the package website on GitHub Pages
whenever you make a package release.
I need to know the URL of the GitHub repository.
It is of the form https://github.com/USER/REPOS.
What is USER (typically your GitHub username)? demo-user
What is REPOS, the repository name? DemoPackage
Shall your package provide a GAP kernel extension? Yes, written in C

Next I will ask you about the package authors and maintainers.

Last name? Doe
First name(s)? Dana
Is this one of the package authors? [Y/n] y
Is this a package maintainer? [Y/n] y
Email? dana@example.invalid
WWWHome? https://example.invalid/~dana/
GitHubUsername? demo-user
PostalAddress? Example Institute\nDepartment of Algebra\nExample Street 1\n12345 Example City
Place? Example City, Country
Institution? Example Institute
Add another person? [y/N] n
Creating the git repository...
Initialized empty Git repository in .../DemoPackage/.git/
[main (root-commit) ...] initial import
Done creating git repository.
Create <https://github.com/demo-user/DemoPackage> via <https://github.com/new> and then run:
git push -u origin main

```

After this finishes, the new package lives in a directory named DemoPackage below the current directory. You can now inspect and edit the generated files, move the package into a package directory, and try loading it in GAP.

1.4.2 PackageWizardGenerate

▷ PackageWizardGenerate(arg)

(function)

1.4.3 PackageWizard

▷ `PackageWizard(arg)` (function)

1.5 Important wizard choices

Some answers mainly affect metadata, while others determine which files are generated.

1.5.1 Package name

The package name becomes the name of the top-level package directory and is also used in file names such as `gap/YourPackage.gd`, `gap/YourPackage.gi` and in many entries of `PackageInfo.g`. It therefore should be chosen with care.

1.5.2 License

The license choice affects both the generated `LICENSE` file and the guidance inserted into `README.md`. By default, the wizard uses `GPL-2.0-or-later`, because that is the license of **GAP** itself and is also used by many **GAP** packages. Other built-in choices are `GPL-3.0-or-later`, `MIT` and `BSD-3-Clause`.

If you are unsure which license to use, then the following sites may help:

- <https://choosealicense.com/>
- <https://opensource.org/licenses/>
- <https://spdx.org/licenses/>

If you choose the custom option, then **PackageMaker** generates a placeholder `LICENSE` file which you must replace with the full text of your chosen license. You should also make sure that the `License` field in `PackageInfo.g` and the package `README.md` describe the same license.

1.5.3 GitHub setup

Saying yes to GitHub setup does two things:

- it fills in repository-related URLs in `PackageInfo.g`;
- it offers to create a local git repository with an initial commit and a GitHub remote named `origin`.

If you also enable GitHub Actions, then **PackageMaker** adds CI and release workflows below `.github/workflows/` and creates `.codecov.yml`. The release workflow updates the package website on GitHub Pages when you make a release.

If you do not use GitHub setup, then `PackageInfo.g` contains placeholder URL entries instead. They work as reminders, but you should replace them before publishing your package.

1.5.4 Kernel extension

The kernel extension choice determines whether the package is purely interpreted GAP code or also contains code in C or C++. Choosing a kernel extension adds files such as `src/YourPackage.c` or `src/YourPackage.cc`, together with `configure`, `Makefile.in` and `Makefile.gappkg`.

1.6 What gets generated?

Every generated package contains the basic files needed for a small but usable package:

- `PackageInfo.g` with the package metadata;
- `README.md` with placeholder text to replace;
- `init.g` and `read.g`;
- `gap/YourPackage.gd` and `gap/YourPackage.gi`;
- `makedoc.g` for building the package manual;
- `tst/testall.g` as a starting point for tests;
- `LICENSE`.

Depending on your choices, the wizard may also create:

- `.gitattributes` and `.gitignore` when GitHub setup is enabled;
- `.github/workflows/CI.yml`, `.github/workflows/release.yml` and `.codecov.yml` when GitHub Actions are enabled;
- `src/`, `configure`, `Makefile.in` and `Makefile.gappkg` when a kernel extension is requested.

A few of these files deserve immediate attention:

- `README.md` still contains `TODO` text and should be rewritten;
- `PackageInfo.g` should be checked carefully, especially the package URLs, description, authors and maintainers;
- `tst/testall.g` is only the test driver, so you will usually want to add actual `.tst` files;
- the generated `gap/` and optional `src/` files are only a starting skeleton and should be adapted to your package.

1.7 What next?

Once the wizard has finished, a typical next-step checklist is:

- decide where the package should live, as explained in [1.3](#);
- edit `README.md` and replace the `TODO` sections;

- review `PackageInfo.g` and fix any remaining placeholder values;
- if you selected the custom license option, replace the placeholder `LICENSE` text with the full license text;
- add package code and tests;
- run the package inside `GAP` and make sure it loads;
- if you asked for a kernel extension, run `./configure` and build it;
- if you enabled GitHub setup, create the GitHub repository and push the generated initial commit.

For broader guidance on package structure and release preparation, it is worth reading the `GAP` manual chapter on (**Reference: Using and Developing GAP Packages**) as well as the manual and `PackageInfo.g` file of the <https://github.com/gap-packages/example> package.

Index

PackageWizard, [6](#)
PackageWizardGenerate, [5](#)
PackageWizardInput, [4](#)